

## Material para apresentação do projeto final

Este documento explica o funcionamento do aplicativo, separando a parte Flutter da parte de IA, com linguagem simples para apresentação ao professor.

| Camada     | Responsabilidade                                                            |
|------------|-----------------------------------------------------------------------------|
| Flutter    | Interface, navegação, login, cadastro, catálogo, produtos, carrinho e chat. |
| SQLite     | Armazena usuários e produtos localmente no dispositivo.                     |
| Backend IA | Recebe perguntas do app e chama a OpenAI com a chave protegida.             |
| OpenAI     | Gera respostas inteligentes usando a pergunta do usuário e o catálogo.      |

## 1. Visão geral do projeto

O projeto Muscleway é um aplicativo Flutter de loja de suplementos. Ele possui login, cadastro, catálogo de produtos, busca, filtros por categoria, página de detalhes, carrinho de compras e um consultor com IA.

A ideia principal é simular um aplicativo real de e-commerce para suplementos esportivos, com visual mais profissional, navegação simples e uma área de inteligência artificial para ajudar o cliente a escolher produtos conforme seu objetivo.

O projeto original foi mantido separado. As melhorias foram feitas na pasta Projeto-final-flutter-melhorado/app\_muscley, para permitir comparação com a versão inicial.

## 2. Arquitetura geral

A arquitetura foi dividida em duas partes principais: o aplicativo Flutter e o backend de IA.

O Flutter cuida da interface, navegação, formulários, catálogo, carrinho e comunicação com o backend.

O backend de IA fica na pasta backend\_ai e é responsável por receber a pergunta do app, chamar a API da OpenAI com segurança e devolver a resposta para o Flutter.

Essa separação é importante porque a chave da OpenAI nunca deve ficar dentro do app Flutter. Se a chave ficar no APK, qualquer pessoa poderia extrair e usar indevidamente.

### 3. Parte Flutter: estrutura do aplicativo

O Flutter foi usado para construir toda a experi ncia visual do app. Ele organiza as telas em p ginas, usa widgets reutiliz veis e mant m o tema centralizado para facilitar manuten  o.

A navega  o   feita por rotas nomeadas. Por exemplo, /home abre a tela principal, /carrinho abre o carrinho e /chat-ia abre a aba do consultor de IA.

Os widgets foram separados para reduzir repeti  o. O bot o principal fica em BotaoWidget, o logo em BrandLogoWidget e o card de produto em ProdutoCardWidget.

### 4. Login, cadastro e sess o

O cadastro cria um usu rio local no SQLite. Antes de salvar, o app valida se o e-mail   v lido, se a senha tem tamanho m nimo e se a confirma  o de senha est  correta.

A senha n o   salva como texto puro. Ela passa por hash SHA-256 no UsuarioDatabase, melhorando a seguran a em compara  o com salvar a senha diretamente.

Quando o login   bem-sucedido, o usu rio   salvo no SessaoController. A Home usa esse controller para exibir uma sauda  o com o primeiro nome do usu rio.

### 5. Banco de dados local

O projeto usa SQLite com a biblioteca sqflite. Existem dois bancos principais: um para usu rios e outro para produtos.

UsuarioDatabase cria a tabela usuarios, cadastra novos usu rios, busca usu rio por login e normaliza o e-mail para evitar duplicidade por letras mai sculas/min sculas.

ProdutoDatabase cria a tabela produtos e insere exemplos de suplementos caso o banco ainda esteja vazio. Esses produtos alimentam o cat logo, filtros e tamb m o contexto enviado para a IA.

### 6. Cat logo, busca e produtos

A Home carrega os produtos do SQLite e exibe em cards. Cada card mostra nome, categoria, descri  o curta, pre o com desconto no Pix e pre o no cart o.

A busca filtra por nome, descri  o e categoria. Isso facilita encontrar itens como whey, creatina, kit ou pr -treino.

As categorias aparecem em chips horizontais. Ao selecionar uma categoria, a lista   filtrada dinamicamente.

## 7. Carrinho de compras

O carrinho é controlado pelo `CarrinhoController`, que mantém uma lista de itens em memória durante a sessão do app.

Cada item possui produto e quantidade. O controller calcula total de itens e valor total usando `fold`.

A tela do carrinho permite aumentar, diminuir e remover itens. Também mostra uma barra de progresso para frete grátis acima de R\$ 150 e um resumo final do pedido.

## 8. Parte de IA: como funciona

A IA foi separada em duas camadas. A primeira é a interface Flutter, onde o usuário digita sua pergunta. A segunda é o backend local, que fica fora do app e chama a OpenAI.

O app envia para o backend a pergunta do usuário e a lista de produtos do catálogo. Assim a IA responde usando os produtos que realmente existem no app.

Se o backend estiver desligado ou a OpenAI não responder, o app usa um fallback local. Esse fallback faz sugestões simples por palavras-chave, para a tela nunca ficar quebrada.

## 9. Backend da IA

O backend fica em `backend_ai/server.dart` e roda com o comando `dart server.dart`.

Ele lê as variáveis do arquivo `.env`: `OPENAI_API_KEY`, `OPENAI_MODEL` e `PORT`.

A rota `/health` serve para testar se o backend está vivo e se a chave foi carregada. A rota `/chat` recebe a pergunta e os produtos, chama a OpenAI e retorna uma resposta em JSON.

No emulador Android, o app acessa o backend pelo endereço `http://10.0.2.2:3001`. Esse endereço representa o localhost do computador visto de dentro do emulador.

## 10. Segurança da chave da OpenAI

A chave da OpenAI não fica no Flutter. Isso é importante porque apps mobile podem ser desmontados e analisados. Se a chave estivesse no código Flutter, ela poderia ser roubada.

Por isso foi criada a pasta `backend_ai`. A chave fica em `backend_ai/.env`, que foi adicionada ao `.gitignore` para não ser enviada ao GitHub.

Em um projeto real publicado, esse backend deveria ficar em um servidor na nuvem. Para apresentá-lo escolar, rodar localmente já demonstra a arquitetura correta.

## 11. Como rodar para apresentação

Terminal 1: entrar na pasta `app_muscley` e rodar `flutter run`.

Terminal 2: entrar na pasta `app_muscley/backend_ai` e rodar `dart server.dart`.

Com os dois terminais ligados, abra a aba Consultor IA no aplicativo e pergunte algo como: Quero ganhar massa muscular, o que você recomenda?

Se a IA real estiver funcionando, a resposta vem da OpenAI. Se houver erro de crédito ou chave, o app ainda retorna resposta local para não travar.

## 12. Roteiro curto para falar ao professor

Este projeto começou como um app Flutter simples e foi evoluído para um protótipo de e-commerce de suplementos chamado Muscleyway.

A parte Flutter foi responsável por interface, navegação, cadastro, login, produtos e carrinho. Usei SQLite para armazenar usuários e produtos localmente.

A parte de IA foi feita com uma arquitetura mais segura: o app não guarda a chave da API. Ele chama um backend local em Dart, e esse backend chama a OpenAI.

Também existe fallback local, então o chat continua funcionando mesmo se a API estiver sem crédito, sem internet ou com o backend desligado.

A principal evolução foi transformar um app básico em uma experiência mais parecida com um produto real, com visual mais profissional e fluxo completo de compra.

## Tabela de arquivos principais

| Arquivo                          | Função no projeto                                                                                               |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------|
| lib/main.dart                    | Ponto de entrada do app. Chama runApp e inicializa o AppMuscley.                                                |
| lib/src/app.muscley.dart         | Configura MaterialApp, tema, título do app e rotas como login, home, carrinho, produto e chat de IA.            |
| lib/src/theme/app_theme.dart     | Centraliza as cores, tema escuro, botões, campos e estilos globais. A identidade usa preto, branco e azul bebê. |
| lib/src/pages/login_page.dart    | Tela de login. Valida e-mail/senha e chama o banco local para autenticar o usuário.                             |
| lib/src/pages/cadastro_page.dart | Tela de cadastro. Valida dados, confirma senha e salva usuário no SQLite.                                       |
| lib/src/pages/home_page.dart     | Tela principal da loja. Mostra campanha, benefícios, busca, categorias e lista de produtos.                     |
| lib/src/pages/produto_page.dart  | Tela de detalhes do produto, com preço no Pix, descrição, benefícios e botão para adicionar ao carrinho.        |
| lib/src/pages/carrinho_page.dart | Tela do carrinho. Permite aumentar, diminuir, remover produtos e finalizar compra.                              |
| lib/src/pages/chat_ia_page.dart  | Tela exclusiva do consultor IA. Permite conversar e pedir sugestões de suplementos.                             |

## Comandos essenciais

Terminal 1 - aplicativo Flutter:

```
cd ...\\app_muscley  
flutter run
```

Terminal 2 - backend da IA:

```
cd ...\\app_muscley\\backend_ai  
dart server.dart
```

Teste do backend:

```
http://localhost:3001/health
```